DOORS rate policy configuration and implementation

Technical reference | 25 September 2026

1 Rate policy

Master API enforces a token bucket limit on requests reaching its rate-limit filter. The browser UI displays a warning when the server returns HTTP 429. The allowance is tracked separately for each caller and request URL path within each Master API instance.

Caller identification

The caller is request.getRemoteUser() when that value is non-null; otherwise it is request.getRemoteAddr(). The bucket key is caller + ":" + request URI. Session IDs, JWT values and API keys are not explicitly used as rate-limit keys. Username-based counting therefore depends on the authenticated username being available at this filter.

Policy	Initial capacity	Continuous refill
UI default	10 requests per caller and path	10 per minute; about 1 token every 6 seconds
Gateway default	5 requests per caller and path	5 per minute; about 1 token every 12 seconds

What counts as a hit

Each request admitted by the filter consumes one token before downstream processing, even if that processing later fails. OPTIONS preflight requests are skipped. A click can issue several HTTP requests. An exhausted bucket rejects the request with HTTP 429; it does not queue the request.

How the allowance recovers

A new bucket starts full. Greedy refill restores tokens progressively up to the capacity. After ten immediate UI requests to the same path, another immediate request is rejected; approximately six seconds later, one request can be admitted if no other request has consumed the replenished token. This is not a strict rolling-window maximum of ten requests in every sixty seconds.

Rate policy examples

What 10 requests per caller and path means

Each caller gets a separate allowance for each API URL path. For example, if the filter identifies the username as alice, her calls to /reports/list use one bucket and her calls to /reports/export use another. Bob has his own bucket for /reports/list when his username is available to the filter.

Caller and path	Default UI allowance
alice calling /reports/list	One bucket with up to 10 tokens
alice calling /reports/export	A separate bucket with up to 10 tokens
bob calling /reports/list	His own bucket with up to 10 tokens

These are illustrative paths. Query parameters do not create a new allowance: /reports/list?page=1 and /reports/list?page=2 use the same path and bucket for a given caller. If usernames are unavailable, callers with the same visible remote IP share that path's bucket.

What 10 immediate requests means

Immediate means API calls sent one after another without waiting, or sent simultaneously. With a full default UI bucket, the same caller can make a burst of 10 calls to the same path without waiting for tokens to refill.

Example sequence	Rate limiter result
First 10 calls in a rapid burst	Allowed by the rate limiter; downstream checks still apply
An 11th call before one token refills	Rejected with HTTP 429 Too Many Requests
About 6 seconds after the rapid burst	One token is available for one more call if no other call consumed it

For simultaneous requests, the limiter admits whichever requests obtain the available tokens; the request numbering above is illustrative. The example assumes a full starting bucket and a burst short enough that no whole token refills during it. A partly depleted bucket allows fewer immediate calls.

API calls are different from clicks and login tokens

One button click may generate several API calls. Each call reaching the limiter is evaluated against its own caller and path bucket, except OPTIONS preflight requests. A bucket token is one unit of request allowance; it is not the session cookie, JWT or API key.

If no further requests consume tokens, an empty default UI bucket refills to 10 tokens in approximately one minute. The default gateway policy instead has 5 tokens and restores about one every 12 seconds. These allowances apply separately within each Master API instance.

2 Configuration

Master API: src/main/resources/application.yml, lines 113–124

The relevant settings are nested under doors.security. The excerpt below omits unrelated settings and comments.

```
doors:
  security:
    rate-limit:
      ui:
        capacity: 10
        refill-minutes: 1
        penalty-minutes: 1
      gateway:
        capacity: 5
        refill-minutes: 1
    gateway-path-pattern: "/v1/master/gateway/orchestrate/"
```

The gateway policy is selected when the request URI contains the gateway-path-pattern string. All other paths reaching this filter receive the UI policy. The policy name does not restrict it to requests made by a human or a browser.

Configuration file	UI capacity	Gateway capacity
application.yml	10	5
application-dev.yml	10	5
application-multidomain.yml	60	30

All listed policies refill over one minute. The effective deployed values depend on active profiles and external configuration overrides.

Property binding

Master API: src/main/java/org/gepnic/doors/masterapi/config/RateLimitProperties.java, lines 7–25

RateLimitProperties uses @ConfigurationProperties(prefix = "doors.security"). Its nested RateLimitConfig holds ui and gateway Policy objects. Each Policy declares capacity, refillMinutes and penaltyMinutes.

The filter reads capacity and refillMinutes. Although penalty-minutes is configured and its YAML comment describes a lockout, the current filter never reads it and implements no separate penalty period.

3 Backend implementation

Master API: src/main/java/org/gepnic/doors/masterapi/config/RateLimitFilter.java

RateLimitFilter is a Spring @Component extending OncePerRequestFilter. The excerpts below preserve the implementation logic; long expressions are wrapped and comments are omitted for readability.

Store counters and identify the caller

Lines 25–26 and 67–70

```
private final RateLimitProperties properties;
private final Map<String, Bucket> buckets =
        new ConcurrentHashMap<>();

private String resolveClientKey(HttpServletRequest request) {
    String authUser = request.getRemoteUser();
    return (authUser != null)
            ? authUser : request.getRemoteAddr();
}
```

Skip preflight and choose the policy

doFilterInternal, lines 34–47

```
if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
    filterChain.doFilter(request, response);
    return;
}

String uri = request.getRequestURI();
String key = resolveClientKey(request);

RateLimitProperties.Policy policy =
        uri.contains(properties.getGatewayPathPattern())
        ? properties.getRateLimit().getGateway()
        : properties.getRateLimit().getUi();

Bucket bucket = buckets.computeIfAbsent(
        key + ":" + uri, k -> createNewBucket(policy));
```

The same username and path share an allowance across sessions and tokens. Different paths have separate allowances. Query parameters and HTTP methods are not included in the key. When username lookup falls back to IP, callers sharing the same visible IP also share the allowance for a path.

4 Token refill and rejection code

Master API: src/main/java/org/gepnic/doors/masterapi/config/RateLimitFilter.java

Create the bucket

createNewBucket, lines 59–64

```
private Bucket createNewBucket(RateLimitProperties.Policy policy) {
    return Bucket.builder()
        .addLimit(Bandwidth.classic(
            policy.getCapacity(),
            Refill.greedy(
                policy.getCapacity(),
                Duration.ofMinutes(policy.getRefillMinutes()))))
        .build();
}
```

Bucket4j supplies the token bucket implementation. The declared dependency is com.github.vladimir-bukhtoyarov:bucket4j-core:7.6.0 in masterapi/pom.xml, lines 139–143. A bucket token means one unit of request allowance, not an authentication token.

Consume one token or return HTTP 429

doFilterInternal, lines 49–55; logging statement omitted

```
if (bucket.tryConsume(1)) {
    filterChain.doFilter(request, response);
} else {
    response.setStatus(429);
    response.setHeader(
        "X-Rate-Limit-Retry-After-Seconds", "60");
    response.getWriter().write(
        "Too many requests. Please wait a moment.");
}
```

A successful consume continues the filter chain. A failed consume ends processing with the response shown above. The retry header is always 60 seconds; it is not calculated from the bucket and does not impose a sixty-second lockout. A replenished token may permit an earlier request.

Counter lifetime and deployment scope

Buckets live in the JVM ConcurrentHashMap. Restarting Master API resets them; separate instances have independent counters. This class has no bucket expiration or eviction. New caller and path combinations therefore create additional retained entries. It does not implement a shared cluster-wide quota or an explicit per-API-client quota.

5 UI implementation and operational interpretation

UI: src/api/axios-instance.ts, lines 79–83

```
if (status === 429) {
  ElMessage.warning('Too many requests. Please wait a moment.');
  return Promise.reject(error);
}
```

The shared Axios response interceptor shows the warning and passes the failure back to the calling screen. It has no rate counter, automatic 429 retry, countdown or request cooldown. The backend remains the enforcement point.

Separate generated SDK behavior

UI: src/views/governance/DocumentServiceRegistry.vue, lines 395–422

This Vue file also contains generated Java document-download SDK code. Its sendDownloadWithRetry method retries 429 responses up to a configured attempt limit. It reads X-Rate-Limit-Retry-After-Seconds, then Retry-After, with a bounded exponential fallback. awaitDownloadPermit spaces download requests. These behaviors belong to the generated Java SDK, not the browser Axios interceptor.

Examples of shared and separate allowances

Requests to the same path	Resulting scope
Same available username from two sessions	Shared bucket
Different available usernames behind one IP	Separate buckets
No username and the same remote IP	Shared bucket
Same caller with different API keys	Shared bucket if resolved caller is unchanged
Same caller with different URL paths	Separate buckets

Deployment checks

Confirm the effective profile values, whether getRemoteUser() is populated at the filter, and which remote address is visible after the proxy configuration. Check whether requests are distributed across multiple Master API instances. These determine the effective identity and scope in a running deployment.

Source reference date: 25 September 2026. This document describes the implementation in the Development-Staging doors-ui and masterapi source trees. It does not certify active production settings or runtime identity resolution.